

ASSIGNMENT-10.3

NAME: G. MANI SHARAN REDDY

BATCH: 41

HALL.NO: 2303A52148

1) Problem Statement 1: AI-Assisted Bug Detection

Scenario:

A junior developer wrote the following Python function to

Prompt:

I am working on a Python program that calculates the factorial of a number. The function contains a logical bug and produces incorrect output.

CODE: before review

```
calculate factorials:      Statements must be separated by semicolons
def factorial(n):
    result = 1             Expected indented block
    for i in range(1, n):  "n" is not defined
        result = result * i Expected indented block
    return result
|
```

CODE: After review

```
10 / ...
calculate factorials:
def factorial(n):
    result = 1
    for i in range(1, n+1):
        result = result * i
    return result

print(factorial(5))
```

Output:

```
PS C:\Users\2303A\OneDrive\2148 projects\ai> & C:/Users/2303A/AppData/Local/Python/pythoncore-3.14-64
/python.exe "c:/Users/2303A/OneDrive/2148 projects/ai/lab-10"
120
```

Problem Statement 2: Task 2 — Improving Readability &

Documentation

Scenario: The following code works but is poorly written:

Prompt:

I am working with a Python function that performs an operation based on user input.

The function currently has poor readability, unclear parameter names, no documentation, and weak error handling

Code: Before review

```
1 def calc(a, b, c):
2     if c == "add":
3         return a + b
4     elif c == "sub":
5         return a - b
6     elif c == "mul":
7         return a * b
8     elif c == "div":
```

Code: After review

```
def perform_calculation(num1, num2, operation):  
    # Validate operation  
    valid_operations = {'add', 'sub', 'mul', 'div'}  
    if operation not in valid_operations:  
        raise ValueError(f"Invalid operation '{operation}'. Must be one of: {'', ' '.join(valid_o  
    # Perform the calculation  
    if operation == "add":  
        return num1 + num2  
    elif operation == "sub":  
        return num1 - num2  
    elif operation == "mul":  
        return num1 * num2  
    elif operation == "div":  
        if num2 == 0:  
            raise ZeroDivisionError("Cannot divide by zero")  
        return num1 / num2  
  
# Test cases  
if __name__ == "__main__":  
    try:  
        print(f"10 + 5 = {perform_calculation(10, 5, 'add')}")  
        print(f"10 - 5 = {perform_calculation(10, 5, 'sub')}")  
        print(f"10 * 5 = {perform_calculation(10, 5, 'mul')}")  
        print(f"10 / 5 = {perform_calculation(10, 5, 'div')}")  
        print(f"10 / 0 = {perform_calculation(10, 0, 'div')}") # This will raise an error  
    except (ValueError, ZeroDivisionError, TypeError) as e:  
        print(f"Error: {e}")
```

Output:

```
PS C:\Users\2303A\OneDrive\2148 projects\ai> & C:/Users/2303A/AppData/Local/Python/  
/python.exe "c:/Users/2303A/OneDrive/2148 projects/ai/10.1.py"  
10 + 5 = 15.0  
10 - 5 = 5.0  
10 * 5 = 50.0  
10 / 5 = 2.0  
Error: Cannot divide by zero  
PS C:\Users\2303A\OneDrive\2148 projects\ai> █
```

Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

Prompt:

I have written a Python function that works correctly for sample inputs but does not follow PEP8 coding standards and has poor readability.

Code : Before review

```
def Checkprime(n):  
    for i in range(2, n):    Expected indented block  
        if n % i == 0:    Expected indented block  
            return False    Expected indented block  
    return True
```

Code : After review

```
def check_prime(number: int) -> bool:  
    # Validate input  
    if not isinstance(number, int):  
        raise TypeError("Input must be an integer")  
  
    # Handle edge cases: numbers less than 2 are not prime  
    if number < 2:  
        return False  
  
    # Optimization: only check divisors up to the square root of the number  
    # If a number has a divisor greater than its square root,  
    # it must also have a divisor smaller than its square root  
    for divisor in range(2, int(number ** 0.5) + 1):  
        if number % divisor == 0:  
            return False  
  
    return True  
  
# Test cases  
if __name__ == "__main__":  
    test_numbers = [1, 2, 3, 4, 5, 10, 17, 20, 23, 100]  
  
    for num in test_numbers:  
        result = check_prime(num)  
        print(f"{num} is {'prime' if result else 'not prime'}")
```

Output:

```
/python.exe "c:/Users/2303A/OneDrive/2148 projects/ai/10.2.py"
1 is not prime
2 is prime
3 is prime
4 is not prime
5 is prime
10 is not prime
17 is prime
20 is not prime
23 is prime
100 is not prime
PS C:\Users\2303A\OneDrive\2148 projects\ai> 
```

Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario:

In a GitHub project, a teammate submits:

Prompt:

I am reviewing a Python function that processes a list of numbers.
Please act as a code reviewer and help improve the quality, safety, and clarity of the function.

Code : before review

```
def processData(d):
    return [x * 2 for x in d if x % 2 == 0]    Expected indented block
```

Code : After review

```

def process_numbers(numbers: list) -> list:
    # Input validation
    if not isinstance(numbers, list):
        raise TypeError(f"Expected list, got {type(numbers).__name__}")

    if not numbers:
        raise ValueError("Input list cannot be empty")

    # Validate that all elements are numeric
    for item in numbers:
        if not isinstance(item, (int, float)):
            raise TypeError(f"All list elements must be numeric, got {type(item).__name__}")

    # Filter even numbers and double them
    result = [num * 2 for num in numbers if num % 2 == 0]

    return result

# Test cases
if __name__ == "__main__":
    test_cases = [
        [1, 2, 3, 4, 5, 6],
        [10, 15, 20, 25],
        [2, 4, 6, 8],
        [1, 3, 5, 7],
        [-2, -1, 0, 1, 2],
    ]

    print("Testing process_numbers function:\n")
    for test in test_cases:
        try:

```

Output:

```
/python.exe "c:/Users/2303A/OneDrive/2148 projects/ai/10.3.py"  
Testing process_numbers function:
```

```
Input:  [1, 2, 3, 4, 5, 6]  
Output: [4, 8, 12]
```

```
Input:  [10, 15, 20, 25]  
Output: [20, 40]
```

```
Input:  [2, 4, 6, 8]  
Output: [4, 8, 12, 16]
```

```
Input:  [1, 3, 5, 7]  
Output: []
```

```
Input:  [-2, -1, 0, 1, 2]  
Output: [-4, 0, 4]
```

Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

prompt:

I am working with a Python function that computes the sum of squares of a list of numbers. The function is correct, but I want to analyze and optimize its performance for large inputs.

Code : Before review

```
def sum_of_squares(numbers):  
    total = 0      Expected indented block  
    for num in numbers:      "numbers" is not defined  
        total += num ** 2    Expected indented block  
    return total
```

Code : After review

```
# Performance benchmark
if __name__ == "__main__":
    from functools import wraps

    def benchmark(func):
        """Decorator to measure function execution time"""
        @wraps(func)
        def wrapper(*args, **kwargs):
            start = time.perf_counter()
            result = func(*args, **kwargs)
            end = time.perf_counter()
            return result, (end - start) * 1000 # Return time in milliseconds
        return wrapper

    # Test data
    small_list = [1, 2, 3, 4, 5, 10, 20]
    medium_list = list(range(1, 10001)) # 10,000 numbers
    large_list = list(range(1, 100001)) # 100,000 numbers

    print("=" * 70)
    print("PERFORMANCE ANALYSIS: Sum of Squares Functions")
    print("=" * 70)

    # Small list test
    print("\n1. SMALL LIST (7 elements):")
    print("-" * 70)

    except ImportError:
        pass

    # Large list test
    print(f"\n3. LARGE LIST ({len(large_list)} elements):")
    print("-" * 70)

    for name, func in methods:
        result, time_ms = benchmark(func)(large_list)
        print(f" {name:25} | Result: {result:20} | Time: {time_ms:.4f}ms")

    try:
        result, time_ms = benchmark(sum_of_squares_numpy)(large_list)
        print(f" {'NumPy':25} | Result: {result:20} | Time: {time_ms:.4f}ms")
    except ImportError:
        pass

    print("\n" + "=" * 70)
    print("PERFORMANCE RECOMMENDATIONS:")
    print("=" * 70)
    print("• Small inputs (<1000):          Use list comprehension (readable & fast)")
    print("• Medium inputs (<100k):          Use list comprehension or map()")
    print("• Large inputs (>100k):          Use NumPy for significant speedup")
    print("• Memory-critical:                Use map() for generator approach")
    print("=" * 70)
```

OUTPUT:

PERFORMANCE RECOMMENDATIONS:

- Small inputs (<1000): Use list comprehension (readable & fast)
- Medium inputs (<100k): Use list comprehension or map()
- Large inputs (>100k): Use NumPy for significant speedup
- Memory-critical: Use map() for generator approach

PS C:\Users\2303A\OneDrive\2148 projects\ai> █